

Double Descent in Linear Models

Andis Bara

University of Milan – MSc. Data Science for Economics and Health

A.Y. 2025/26

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Usually in classical statistical learning theory, model complexity is related to the bias-variance trade-off. A model that is too simple seems to underfit the data, while a model that is too complex may do the opposite, overfit and perform badly on unseen data. This produces the classic U-shaped test error curve. In these cases test error first seems to decrease as the model becomes more complex and then increases the moment the model starts to fit patterns that are not truly part of the true data-generating process.

However, results from modern machine learning suggest that this classical picture is not always sufficient. When the model is over parametrized the test error in this case does not seem to increase when the model is beyond the usual overfitting zone but, instead after the model reaches the interpolation threshold, the test error begins to drop again and this behavior is referred to as *double descent*.

On this project I study this behavior in a more simpler way, using linear regression. I keep the number of observations fixed and vary the number of dimensions, then the comparison is made between two linear methods: ordinary least squares and ridge regression. The aim is to show how the training and test error move before the interpolation threshold, around it, and after the model is overparameterized.

2 Background: Double Descent

The data is represented by a feature matrix

$$X \in \mathbb{R}^{n \times d},$$

in which n is the number of observations and d is the number of features. Increasing d means increasing the number of parameters that the linear model can use.

A linear regression model predicts the target variable as

$$\hat{y} = X\hat{w},$$

where $\hat{w} \in \mathbb{R}^d$ is the estimated vector of coefficients.

The interpolation threshold appears when the size of the model's parameters is roughly on the same scale as the number of training observations. For the linear model considered here, this happens around the point where the number of features d catches up with the sample size n , so approximately

$$d \approx n.$$

This point is slightly "delicate". The model is no longer clearly underparameterized, but it is also not yet safely in the overparameterized region. It has just enough freedom to fit the training sample very closely, including parts of the sample that come from noise rather than from the true signal. For that reason, the behavior around this threshold can be unstable. Under the usual bias-variance intuition, increasing the number of parameters beyond this point should only make the situation worse. The double descent view complicates this story: after the interpolation peak, adding more dimensions can actually reduce the test error again.

In this project, I use the following distinction between the three regions:

$$d < n \quad \text{underparameterized regime,}$$

$$d \approx n \quad \text{interpolation threshold,}$$

$$d > n \quad \text{overparameterized regime.}$$

The expected double descent curve therefore exhibits a classical descent, a peak around interpolation, and then a second descent in the overparameterized region.

3 Data Generation

The dataset used in this experiment is synthetic. This choice is useful because it allows the data-generating process to be fully controlled. The target variable is generated from a noisy linear model:

$$y = Xw^* + \varepsilon,$$

where X is the feature matrix, w^* is the true coefficient vector, and ε is a Gaussian noise term.

The noise term is generated as

$$\varepsilon \sim \mathcal{N}(0, \sigma^2),$$

where σ is controlled by the parameter `noise_std`. In the main experiment, the noise standard deviation is set to

$$\sigma = 0.5.$$

The feature matrix is not regenerated from zero for every value of d . Instead, I first generate a full matrix with a maximum number of features, denoted by $d_{\max}$. After that, each experiment only takes a subset of this matrix, namely the first d columns. In this way, the underlying random sample stays fixed while the visible number of features changes. This makes the comparison across dimensions cleaner, because the main thing changing from one run to another is the model dimension, not the whole dataset itself.

The true weight vector is defined as

$$w^* \in \mathbb{R}^{d_{\max}}.$$

Only the first d_{true} components of w^* are non-zero, while the remaining components are set equal to zero. Formally,

$$w_j^* \neq 0 \quad \text{for } j \leq d_{\text{true}},$$

and

$$w_j^* = 0 \quad \text{for } j > d_{\text{true}}.$$

This means that only the first features contain the real signal. The remaining features are irrelevant for the true data-generating process, but they can still be used by the fitted model as the dimension d increases.

The main parameters used in the experiment are:

$$n_{\text{train}} = 100,$$

$$n_{\text{test}} = 1000,$$

$$d_{\max} = 300,$$

$$d_{\text{true}} = 10.$$

The test set is larger than the training set because it is only used to estimate generalization error more smoothly. The interpolation threshold still depends on the training size, so it is expected around

$$d = n_{\text{train}} = 100.$$

4 Models

Two linear models are implemented from scratch using NumPy for matrix computations. No high-level machine learning fitting function is used.

4.1 Least Squares Regression

The first model is ordinary least squares. The objective is to find the coefficient vector w that minimizes the squared error on the training data:

$$\min_w \|Xw - y\|^2.$$

When $d < n$, this is the standard underparameterized least squares problem. When $d > n$, there may be infinitely many solutions that fit the training data. In that case, the Moore–Penrose pseudoinverse selects the minimum-norm solution.

The solution is written as

$$\hat{w}_{LS} = X^+ y,$$

where X^+ denotes the Moore–Penrose pseudoinverse of X .

This solution is important in the overparameterized regime because it does not choose an arbitrary interpolating solution. Instead, among all solutions that fit the training data, it chooses the one with the smallest Euclidean norm. This is connected to the idea that some interpolating solutions may generalize better than others.

The fitted predictions are then computed as

$$\hat{y} = X\hat{w}_{LS}.$$

4.2 Ridge Regression

The second model is ridge regression. Ridge regression modifies least squares by adding an L_2 penalty on the size of the coefficients. Its objective function is

$$\min_w \|Xw - y\|^2 + \alpha \|w\|^2,$$

where $\alpha > 0$ controls the strength of regularization.

The closed-form solution is

$$\hat{w}_{Ridge} = (X^T X + \alpha I)^{-1} X^T y.$$

Here, I is the identity matrix of dimension $d \times d$. The identity matrix appears because the L_2 penalty adds α to the diagonal of $X^T X$. In the implementation, the inverse is not computed explicitly. Instead, the linear system

$$(X^T X + \alpha I)\hat{w}_{Ridge} = X^T y$$

is solved directly.

The value used in the main experiment is

$$\alpha = 10^{-1}.$$

Ridge regression is included as a comparison because regularization should reduce instability near the interpolation threshold. In other words, while least squares may have a sharp test-error peak around $d = n$, ridge is expected to produce a smoother curve.

5 Evaluation Metric

The performance of each model is evaluated using mean squared error. For a set of observations $(y_i, \hat{y}_i)$, the mean squared error is defined as

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2,$$

where m is the number of observations being evaluated.

Both training MSE and test MSE are computed. Training MSE shows how well the model fits the data it was trained on. Test MSE is more important for this project because it measures generalization to new data.

For each value of d , the following steps are performed:

1. Select the first d columns of the full training and test matrices.
2. Fit least squares regression.
3. Compute training and test MSE for least squares.
4. Fit ridge regression.
5. Compute training and test MSE for ridge regression.
6. Store the results for plotting.

The set of dimensions used is concentrated around the interpolation threshold:

$$d \in \{5, 10, 20, 40, 60, 80, 90, 95, 99, 100, 101, 105, 110, 120, 150, 200, 250, 300\}.$$

More values are included around $d = 100$ because that is where the most unstable behavior is expected.

6 Results

6.1 Least Squares

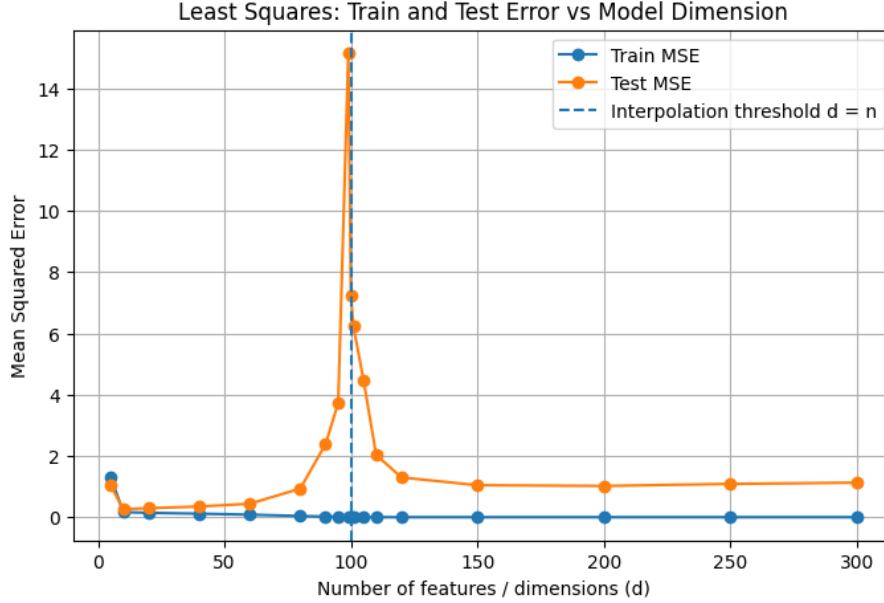


Figure 1: Training and test mean squared error for the least squares model as the number of features d increases. The dashed vertical line marks the interpolation threshold $d = n_{\text{train}} = 100$.

The least squares curve shows the expected behavior around the interpolation threshold. For small values of d , the model is not expressive enough to fully capture the target function, especially because only a limited number of features are available. As the number of dimensions increases, the model becomes more flexible.

Around the point

$$d = n_{\text{train}} = 100,$$

the test error increases very sharply. This is the part of the experiment where the model is close to interpolation: it has enough parameters to fit the training data almost perfectly. At first this may sound like a good thing, because the training error becomes extremely small. But in fact this is also where the model is most fragile. It is not only fitting the useful linear signal; it can also adjust itself to the random noise that was added to the training labels. As a result, the training error is almost zero, while the test error becomes much larger.

After this threshold, when

$$d > n,$$

the test error decreases again. This is the second descent. The model is still able to interpolate the training data, but the minimum-norm solution selected by the pseudoinverse becomes less unstable than the solution near the threshold. The test error does not necessarily keep decreasing forever, but the fall after the peak is the key pattern.

A careful interpretation is therefore needed. The result does not mean that zero training error automatically causes good generalization. Around the interpolation threshold, zero or

near-zero training error is actually associated with poor test performance. The improvement appears after moving further into the overparameterized regime.

6.2 Ridge Regression

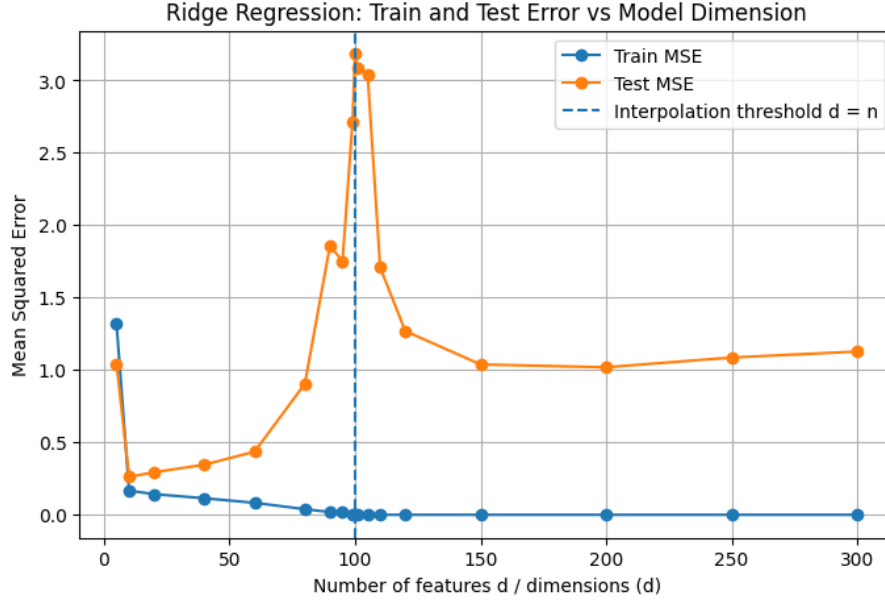


Figure 2: Training and test mean squared error for ridge regression with $\alpha = 0.1$.

Ridge regression behaves more smoothly than ordinary least squares. Because of the regularization term,

$$\alpha \|w\|^2,$$

the model is discouraged from using very large coefficients. This helps reduce the instability that appears near the interpolation threshold.

Compared with least squares, ridge regression is expected to have a smaller or less visible spike around $d = n$. The penalty prevents the model from fitting the training data in an excessively unstable way. This does not mean that ridge always has the lowest test error for every dimension, but it usually provides a more controlled curve.

This result is consistent with the general role of regularization in statistical learning. Ridge regression sacrifices some training fit in order to obtain more stable predictions on unseen data.

6.3 Comparison of Test Errors

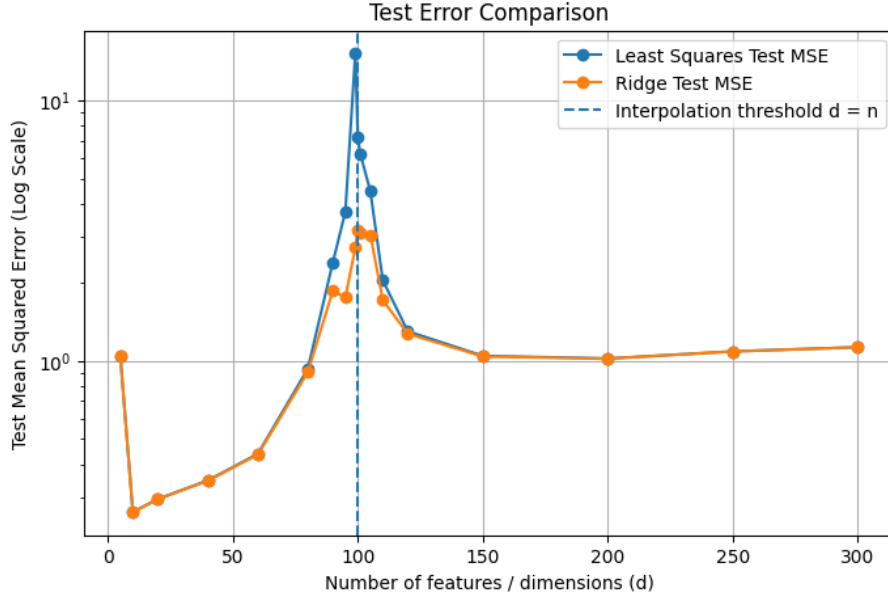


Figure 3: Comparison of test mean squared errors for least squares and ridge regression on a logarithmic scale.

The comparison between least squares and ridge regression highlights the role of interpolation. Least squares is the model that most clearly shows the double descent pattern. Its test error increases sharply near $d = n$, then decreases after the model enters the overparameterized regime.

Ridge regression, on the other hand, reduces the sharpness of the interpolation peak. This happens because the regularization term changes the effective complexity of the fitted model. Even when the number of dimensions increases, ridge does not behave exactly like an unregularized interpolating estimator.

In the comparison plot, the logarithmic scale is useful because the least-squares test error near the interpolation threshold can be much larger than the other values. Without a log scale, the smaller values before and after the peak may look almost flat, even though their differences are still meaningful.

7 Discussion

The experiment provides evidence of double descent in a simple linear regression setting. This is important because double descent is sometimes associated mainly with neural networks, but the phenomenon can also be observed in much simpler models. In this project, the model is linear, and the only changing factor is the number of input dimensions.

The main observation is that increasing model complexity does not lead to a single U-shaped curve. Instead, the test error first follows the classical intuition, then becomes very large near the interpolation threshold, and finally decreases again when the model becomes overparameterized.

This result is connected to the argument of Belkin et al. [1]. The interpolation threshold is not the end of the story. Once the model has enough capacity to interpolate the training data,

increasing capacity further can sometimes lead to solutions with better generalization properties.

Still, the experiment has limitations. First, the data is synthetic, so the results are easier to control than in a real-world dataset. Second, the result depends on the noise level, the number of true relevant features, and the random seed.

Another limitation is that the features are generated independently from a standard normal distribution. This makes the setup clean, but real datasets often contain correlations, non-linearities, and measurement issues. For that reason, the result should be interpreted as an empirical illustration of the double descent mechanism rather than as a universal claim about every dataset.

8 Conclusion

This project studied double descent in linear models by fixing the number of training observations and increasing the number of features. The experiment used a synthetic noisy linear regression dataset and compared ordinary least squares with ridge regression.

The least-squares model showed a clear interpolation peak around

$$d = n_{\text{train}} = 100.$$

Near this threshold, the training error became almost zero, but the test error increased sharply. After moving into the overparameterized regime, the test error decreased again, showing the second descent pattern.

Ridge regression produced a more stable behavior because of the L_2 penalty on the coefficients. This confirmed the role of regularization in controlling instability around the interpolation threshold.

Overall, the results support the idea that the classical bias-variance trade-off does not fully describe what happens in overparameterized models. Even in a simple linear setting, increasing model complexity beyond interpolation can improve test performance.

References

- [1] M. Belkin, D. Hsu, S. Ma, and S. Mandal. *Reconciling modern machine learning practice and the bias-variance trade-off*. Proceedings of the National Academy of Sciences, 2019.
- [2] N. Cesa-Bianchi. *Statistical Methods for Machine Learning: Experimental Projects*, A.Y. 2025/26. Course project assignment document.